

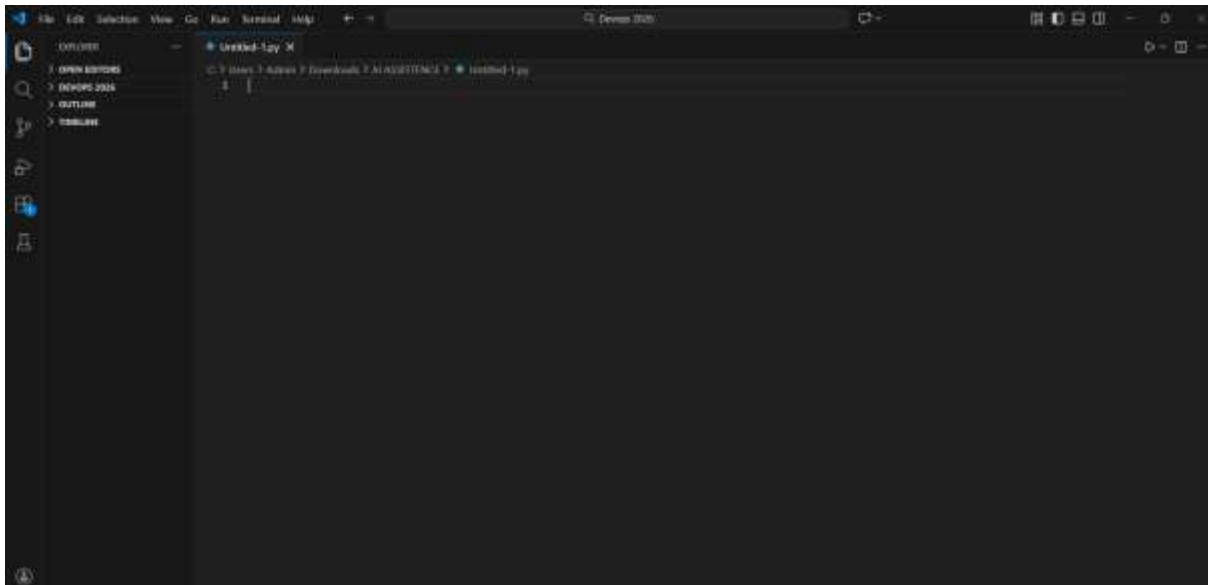
AIAC

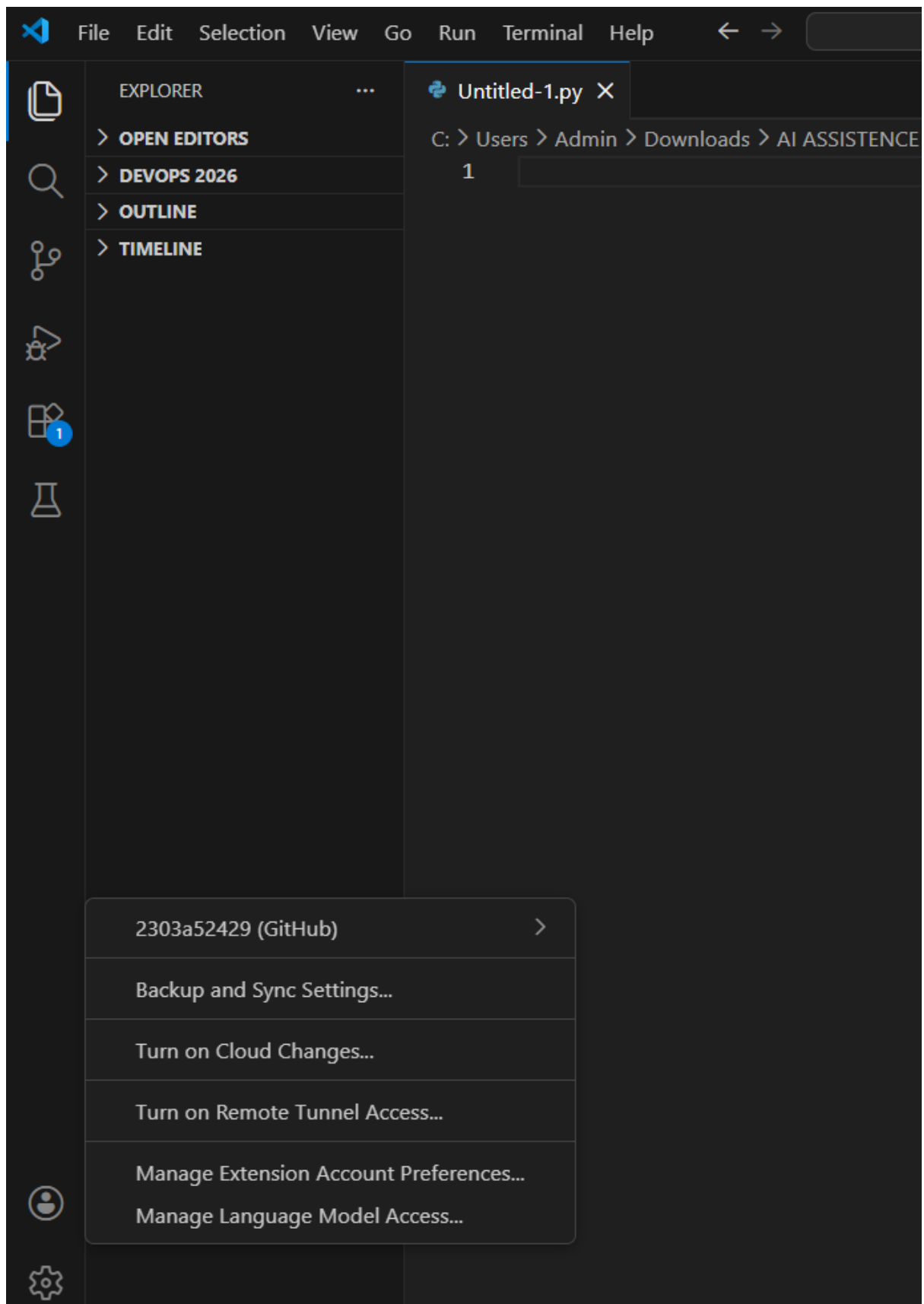
ASSIGNMENT-1.2

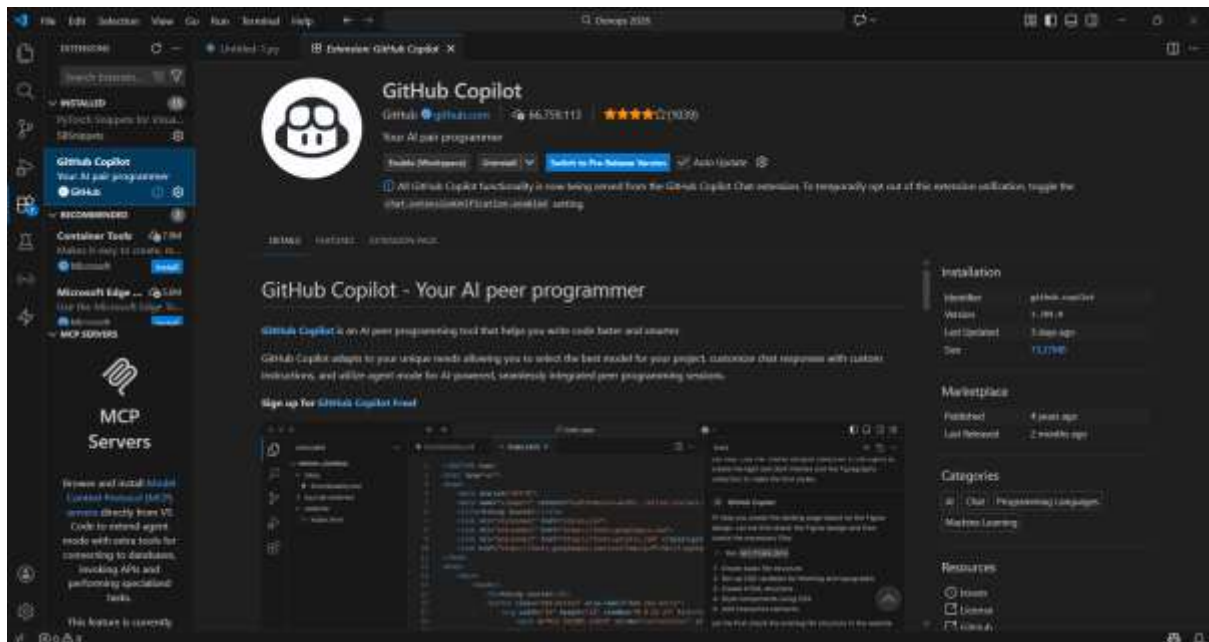
2303A52429

BATCH-32

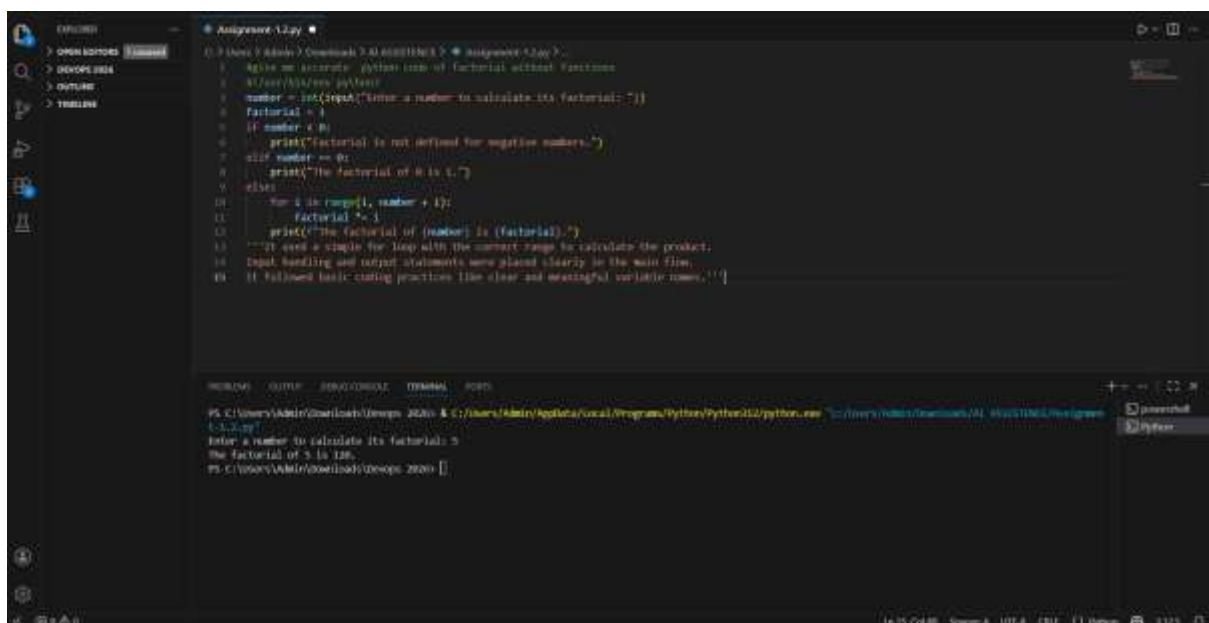
TASK-0







TASK-1



TASK-2

```
File Edit Selection View Go Run Terminal Help DevOps 2026
Assignment 1.2.py X
20 # Give me accurate, python code of factorial and optimize the code, make it more readable
21 number = int(input("Enter a number to calculate its factorial: "))
22 factorial = 1
23 if number < 0:
24     print("Factorial is not defined for negative numbers.")
25 elif number == 0:
26     print("The factorial of 0 is 1.")
27 else:
28     for i in range(2, number + 1): # Start from 2 as multiplying by 1 is redundant
29         factorial *= i
30     print("The factorial of {number} is {factorial}.")
31 # This code calculates the factorial of a given non-negative integer.
32 # It handles edge cases for negative numbers and zero.
33 # The loop starts from 2 to optimize the multiplication process.
34 # The code is structured for clarity and maintainability.
35
```

```
PS C:\Users\Admin\Downloads\DevOps_2026> & C:\Users\Admin\AppData\Local\Programs\Python\Python312\python.exe "C:\Users\Admin\Downloads\AI_AssistENCE\Assignment 1.1.2.py"
Enter a number to calculate its factorial: -2
Factorial is not defined for negative numbers.
PS C:\Users\Admin\Downloads\DevOps_2026>
```

TASK-3

```
File Edit Selection View Go Run Terminal Help DevOps 2026
Assignment 1.2.py X
36 """
37 Give me a code for factorial using functions and use meaningful variable names and meaningful function and give me optimized code
38 def calculate_factorial(n):
39     """Calculate the factorial of a non-negative integer n."""
40     if n < 0:
41         raise ValueError("Factorial is not defined for negative numbers.")
42     factorial_result = 1
43     for i in range(2, n + 1): # Start from 2 as multiplying by 1 is redundant
44         factorial_result *= i
45     return factorial_result
46
47 def main():
48     try:
49         user_input = int(input("Enter a non-negative integer to calculate its factorial: "))
50         result = calculate_factorial(user_input)
51         print(f"The factorial of {user_input} is {result}.")
52     except ValueError as e:
53         print(e)
54
55 if __name__ == "__main__":
56     main()
57
```

```
# This code defines a function to calculate the factorial of a non-negative integer.
# It includes error handling for negative inputs and uses meaningful variable and function names.
# The main function handles user input and output, ensuring clarity and separation of concerns.
```

```
Assignment 1.2.py
Enter a non-negative integer to calculate its factorial: 7
The factorial of 7 is 5040.
PS C:\Users\Admin\Downloads\DevOps_2026>
```

TASK-4

```
File Edit Selection View Go Run Terminal Help DevOps 2026
Assignment 1.2.py X
58 # Give me a comparative table for both the code i.e., using functions and without using functions
59
60 | Aspect | Without Functions | With Functions |
61 |---|---|---|
62 | Structure | Linear flow of code | Modular with separate functions |
63 | Readability | Less readable due to lack of modularity | More readable with clear function definitions |
64 | Reusability | Code cannot be reused easily | Functions can be reused in other parts of the program |
65 | Error handling | Basic error handling within main flow | Improved error handling with exceptions |
66 | Optimization | Basic optimization by starting loop from 2 | Same optimization with clear function logic |
67 | Variable naming | Basic variable names | Meaningful and descriptive variable names |
68 | Input handling | Handling in main flow | Handled in main function |
69 | Output handling | Handled in main flow | Handled in main function |
70
71 # The table compares the two approaches in terms of structure, readability, reusability, error handling, optimization,
72 # variable naming, and input/output handling.
73 # The non-function version was simple but harder to maintain, while the function-based version improves readability,
74 # structure, error handling, and reusability, making the code cleaner and easier to manage.
75
```

TASK-5

```
EXPLORE
> OPEN EDITOR
> DEVOPS 2026
> OUTLINE
> TABLE

Assignment 1.2.py
1: 3 Ques 1 Answer 1 Downloads 1 ALASTAIR2.7 Assignment 1.2.py 2.
2:
3: # Iterative model
4: # Iterative model
5: def iterative_factorial(n):
6:     """Calculate the factorial of a non-negative integer n using an iterative approach."""
7:     if n < 0:
8:         raise ValueError("Factorial is not defined for negative numbers.")
9:     factorial_result = 1
10:    for i in range(2, n + 1):
11:        factorial_result *= i
12:    return factorial_result
```

```
# Recursive model
def recursive_factorial(n):
    """Calculate the factorial of a non-negative integer n using a recursive approach."""
    if n < 0:
        raise ValueError("Factorial is not defined for negative numbers.")
    if n == 0 or n == 1:
        return 1
    return n * recursive_factorial(n - 1)

def main():
    try:
        user_input = int(input("Enter a non-negative integer to calculate its factorial: "))
        iterative_result = iterative_factorial(user_input)
        recursive_result = recursive_factorial(user_input)
        print(f"Iterative: The factorial of {user_input} is {iterative_result}.")
        print(f"Recursive: The factorial of {user_input} is {recursive_result}.")
    except ValueError as e:
        print(e)

if __name__ == "__main__":
    main()
```

```
main()
# This code provides both iterative and recursive implementations of the factorial function.
# It includes error handling for negative inputs and uses meaningful variable and function names.
# The main function handles user input and output for both methods, ensuring clarity and separation of concerns.
# The iterative approach uses a loop to calculate the factorial, while the recursive approach calls itself to compute the result.
# Both methods are encapsulated in functions for reusability and clarity.
```

```
Enter a non-negative integer to calculate its factorial: 25
The factorial of 25 is 15511210043330985984000000.
Enter a non-negative integer to calculate its factorial: 23
Iterative: The factorial of 23 is 25852016738884976640000.
Recursive: The factorial of 23 is 25852016738884976640000.
PS C:\Users\Admin\Downloads\Devops 2026>
```